

➤ Why Commenting

- Comments are for **communicating any information** to future developers who are either using or maintaining the code, which is not obvious from reading the code.
- Comments are an important part of writing code in whatever programming or scripting language you use. Both writing no comment and writing too much comment is bad.
- Due to time restrictions or lazy programmers or enthusiastic programmers who want immediate results for their code, commenting of code often takes a back seat. Programmers working as a team have found it better to leave comments behind since coding usually follows cycles, or more than one person may work on a particular module. However, some commenting can decrease the cost of knowledge transfer between developers working on the same module.
- Comments are a central part of professional coding practice, **careful commenting** will make your **code easier to read and maintain**, impress your employer and fellow coders and boost your reputation as a good coder '**who does it the Right Way**'.

➤ Comments should include

- When writing comments, consider the situations facing a new developer who is unfamiliar with your code, and your present mindset.
- As a general rule of thumb, take some time to pause and reflect before writing. Ask yourself what is most confusing about the program and how can you best explain it in “dummy” language? Also consider why you’re writing the code exactly.
- So when trying to decide what kind of comments to put in your code, take several steps back from the code itself and think about all the problems you've faced, the assumptions you've made, the compromises you've made, etc.
- Comments are the best place to document those things which might warn future developers away from making mistakes, or even just let them know that you've already considered the benefits/limitations of your solution (or even if you've found potential issues with someone else's existing code and haven't been able to fix it).
- Always write why you are writing this piece of code, because this information is not visible until you write them in comments and this is critical to identify any bug or

behavior with changing business environment.

➤ **Commenting Guidelines**

- **one good commenting practice is to leave a brief description of the following:**

- Name of the module.
 - Purpose of the Module.
 - Description of the Module (In brief).
 - Original Author

- **Inline Commenting**

- Inline comments are limited to single-line content. The most beneficial use is a simple-minded explanation for small functionality.
 - When you need to include a large explanation generally a single liner won't do the trick. Descriptive blocks are most notably seen around functions and library files. Whenever you setup a new function it is good practice to add a descriptive block above the declaration.

- **Avoid Obvious Comments**

- Commenting your code is fantastic; however, it can be overdone or just be plain redundant. Take this example:

```
// get the country code
$country_code = get_country_code($_SERVER['REMOTE_ADDR']);

// if country code is US
if ($country_code == 'US') {

    // display the form input for state
    echo form_input_state();
}
```

- When the text is that obvious, it's really not productive to repeat it within comments.
 - If you must comment on that code, you can simply combine it to a single line instead:

```
// display state selection for US users
```

```
$country_code = get_country_code($_SERVER['REMOTE_ADDR']);  
if ($country_code == 'US') {  
    echo form_input_state();  
}
```

- Adding a comment at the beginning of each block of code also emphasizes the visual separation.

- **Comments are not an art project**

```
+-----+  
| Module Name: CalcStock          |  
| Module Purpose: Calculating Stock |  
| Inputs: Prd code, Date Range    |  
| Outputs: Stock Amount           |  
+-----+
```

- It may be a waste of time. The trouble is that it creates a time-wasting maintenance task imposed on anyone working with the code in the future just to keep the pretty little box intact when the text ruins the symmetry of it.

- **Comment While Coding**

- Nobody wants to go back over their program after it's working and document every piece. Most of us don't even want to go back and document the confusing areas. It really takes a lot of work.
- But if you can write the comments while you're coding everything will still be fresh in your mind. This would be the best time to leave open and honest comments about your code.
- Additionally, this will give you practice to getting used to commenting all of your files. The amount of time required to go back and figure out how something works is much larger after you've already built the function. Both your future self and your teammates will thank you for leaving comments ahead of time.
- All time saved by not commenting during the coding process is made up **more than twice at least** by inserting comments and providing documentation after the coding process is finished.

- And at last you can give your code to fellow developer to understand as part of code review and ask him how much he understands it.